

VARUNA

Dynamic AI Traffic Flow Optimizer & Emergency Grid

Comprehensive Project Documentation

Project Overview

An intelligent traffic management system designed to detect accidents in real-time using YOLOv8 deep learning models and automatically optimize traffic signals to reduce congestion and improve emergency response times. Built for multi-way intersections (2-6 way junctions) with smart signal control algorithms optimized for Indian traffic patterns.

Key Information

Repository: <https://github.com/Manas239/VARUNA>

Built For: Indian Innovates Hackathon

Demo Video: <https://www.youtube.com/watch?v=X0kgA0yyB0k>

Status: Production Ready with YOLOv10 Integration In Progress

TABLE OF CONTENTS

1. Project Overview & Vision.....	Page 1
2. Architecture & System Design.....	Page 2
3. Core Components & Features.....	Page 2
4. AI Detection System.....	Page 3
5. Smart Signal Algorithms.....	Page 3
6. Technology Stack & Requirements.....	Page 4
7. System Installation & Setup.....	Page 4
8. Real-World Use Cases & Scenarios.....	Page 5
9. Future Enhancements & Roadmap.....	Page 5
10. Interview Questions & Answers.....	Page 6

SECTION 1: PROJECT OVERVIEW & VISION

Problem Statement

Traffic congestion in urban areas causes delays, economic losses, and impacts emergency response times. Traditional fixed-time traffic signals cannot adapt to real-time conditions or accidents.

Solution Approach

VARUNA uses computer vision and AI to detect traffic incidents and dynamically adjust signals. The system prioritizes emergency vehicles and diverts traffic during accidents.

SECTION 2: ARCHITECTURE & SYSTEM DESIGN

System Architecture - Three Layer Model

Layer 1: Perception/Detection (Backend)

- Video input from cameras/mobile streams
- YOLOv8 deep learning models for object detection
- Detection types: vehicles, accidents, fire, ambulances, damage

Layer 2: Decision Engine (Signal Algorithms)

- 5 core algorithms for different scenarios
- Multi-junction support (2-6 way intersections)
- Incident-aware traffic optimization

Layer 3: Visualization & Control (React Dashboard)

- Real-time live video feed with overlays
- Signal status monitoring at multiple junctions
- Interactive incident history and evidence storage

Data Flow

Camera -> YOLO Detection -> Signal Algorithm -> Traffic Lights

-> Dashboard WebSocket Update -> User Visualization

SECTION 3: CORE COMPONENTS & FEATURES

1. AI Detection Models

YOLOv8 Based Custom Models:

- accident_v2.pt: Detects vehicle crashes and accidents
- vehicle_counting.pt: Counts vehicles per lane
- ambulance_detection.pt: Identifies emergency vehicles
- fire_detection.pt: Detects fire and smoke
- damage_assessment.pt: Analyzes vehicle damage

2. Smart Signal Control Algorithms

SECTION 4: AI DETECTION SYSTEM

Detection Pipeline

Step 1: Video Input

- Camera stream URL (e.g., IP camera or mobile camera app)
- Optimized for 640x480 resolution at 30fps

Step 2: YOLO Processing

- Confidence threshold: 0.15 (flexible filtering downstream)
- Object tracking enabled to maintain consistent IDs
- Persistent tracking across frames

Step 3: Accuracy Filtering

- Fire: minimum 60% confidence + 20x20px box size
- Accident: minimum 50% confidence
- Vehicles: minimum 30% confidence

Step 4: Stabilization

- 12-frame history buffer prevents false positives
- Requires 60% consistent detection across history
- Reduces noise from single-frame glitches

Demo Failsafes

- Motorcycles/suitcases mapped to accident for testing
- Buses/trucks mapped to ambulance simulation

SECTION 5: SMART SIGNAL ALGORITHMS

Algorithm Decision Hierarchy

1. Check for Fire:

Use ALG-4 (Fire Evacuation) - Opens all safe lanes

2. Check for Multiple Accidents (2+):

Use ALG-5 (Multi-Accident) - Route through safest exit

3. Check for Single Accident:

Use ALG-2 (Accident Diversion) - Prioritize opposite lane

4. Normal Conditions:

Use ALG-1 (Adaptive) - Traffic-weighted decision

Cycle Timing

BASE_CYCLE_TIME:

8 seconds per lane during normal operations

EMERGENCY_CYCLE_TIME:

3 seconds for fast switching during incidents

Traffic Thresholds

LOW: <5 vehicles total -> Round-robin mode

HIGH: >15 vehicles -> Weighted priority mode

NORMAL: 5-15 vehicles -> Adaptive proportional

SECTION 6 & 7: TECHNOLOGY & INSTALLATION

Technology Stack

Backend:

- FastAPI (Python web framework)
- Uvicorn (ASGI server)
- YOLOv8 Ultralytics (object detection)
- OpenCV (image processing)
- WebSocket (real-time communication)

Frontend:

- React 18 (UI framework)
- WebSocket client for live updates
- Leaflet.js for GPS mapping

Requirements

- Python 3.8+ with pip package manager
- Node.js 14+ with npm
- Hardware: Intel i5+, 8GB RAM minimum
- Stable internet connection
- IP camera or mobile camera with HTTP stream

Quick Start Methods

Method 1: One-Click (Windows)

Double-click START_SYSTEM.bat

Method 2: Python Script

python start_system.py

Method 3: Manual Setup

Terminal 1: python main.py (Backend on :8000)

Terminal 2: npm install && npm start (Frontend on :3000)

Configuration

Edit main.py:

CAMERA_LAT/LON = your_location_coordinates

TELEGRAM_BOT_TOKEN = for alert notifications